

Blacksmith vs. GitHub Actions

Six real CI jobs from the Immich monorepo, run cold and warm on both platforms — in one sitting, with a lot of help from agents.

Joff Redfern · July 2026 · a personal pilot, not a powered study

TL;DR

- Blacksmith was faster on **every one of the six jobs** I tested — **1.4x to 5.3x**. The standout is ESLint: ~5.3x faster (390–444s on GitHub Actions vs 73–101s on Blacksmith), and it held across three runs.
- Reliability was a genuine tie: **0 platform failures in 14 job-runs on each side**.
- The setup friction is real — a GitHub org, an overnight approval, workflow edits — but it's a **one-time tax** on a tool that then pays out on every CI run.
- What actually worries me isn't the product. It's that **Blacksmith is invisible to the engineers it's helping** — everything surfaces through GitHub's UI, and a tool nobody can see is a tool a budget review can cut without knowing what it does.
- Honest caveat: n=1–2 per cell. This is a real pilot, directional — not a statistically powered study.

How I picked the benchmark

I didn't want a toy repo. I set the evaluation criteria first: real Docker builds, a real test suite, linting, dependency caching, multiple parallel CI jobs, and enough complexity to actually stress both platforms. Then I had my coding agent vet candidate open-source repos against that list before I committed to one. We landed on **immich-app/immich** — a mature, actively-maintained self-hosted photo-management app with a real TypeScript monorepo, Docker multi-arch builds, Postgres integration tests, and a Svelte web frontend. It checks every box, and its CI is heavy enough that runner performance actually matters.

The setup: I forked Immich into a fresh org and ran the same six jobs on two branches — `bench/gha` on stock GitHub-hosted runners, `bench/blacksmith` on Blacksmith — each cold-cache and warm-cache. One honest scope note: Immich's real CI depends on an org-only GitHub App token and self-hosted runner labels, so I wrote a minimal from-scratch workflow (~70 lines) rather than porting their `test.yml` line-by-line. That means this measures runner performance well, but it isn't a dress rehearsal for migrating an existing workflow. And again: `n=1-2` runs per cell. Directional, not p95-grade.

How it actually got done: eight agents in one sitting

This is the part I most want colleagues to notice, because it's the reason this project took an evening instead of a week. Over the course of the pilot I forked off **eight separate agents**, each with a focused job, while the main thread handled all the actual git and CI orchestration directly:

- **Two** to stand up the infrastructure — forking and preparing the repos, and designing the results page.
- **Four running in parallel** to independently research and draft the workflow configs for the four heaviest CI jobs. Each one read the real Immich source to figure out the right commands — Postgres services, submodules, build steps — rather than guessing from the README.
- **Two** successive passes writing this report itself.

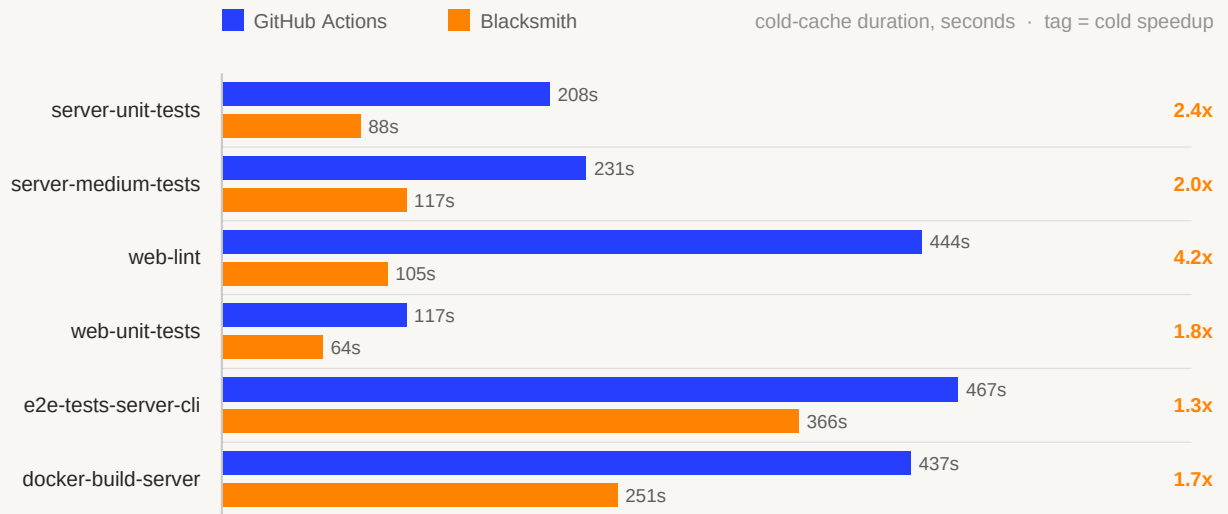
The four parallel drafting agents are the interesting case: writing benchmark workflows for an unfamiliar half-million-line monorepo is exactly the kind of work that serializes badly for one person. Fanned out, it happened in roughly the time it takes to draft one. That's the point to land — agent orchestration wasn't a garnish on this project, it was the enabler.

It wasn't flawless, and that's part of the story

Two incidents worth owning. First, one drafting agent defaulted the Docker job to `push: true` against a public GHCR registry — a public artifact nobody asked for. A safety check blocked it on the Blacksmith branch before it landed; the GitHub Actions branch had already started pushing, and that in-flight run got cancelled mid-push. Both branches were fixed to build-only. Second, a genuine bug: `server-medium-tests` failed identically on both platforms because our checkout wasn't fetching the `test-assets` git submodule — every EXIF-derived test got fallback data. One line (`submodules: 'recursive'`) fixed it on both branches, and everything was green afterward. Neither incident was a platform failure — both were ours, and both got caught.

Results: Blacksmith won every job

Same commit, same workflow logic, same jobs — the only variable is the runner. Blacksmith was faster on all six, from ~1.4x on the e2e suite to a 5.3x gap on the ESLint step. No job favored GitHub Actions.



Cold-cache runs shown; server-medium-tests uses the post-submodule-fix runs on both sides. Full cold+warm data: results/pilot-results.csv.

Job	GHA (all runs)	Blacksmith (all runs)	Speedup
server-unit-tests	167–208s	83–101s	~2.1x
server-medium-tests	216–231s	117–122s	~1.9x
web-lint	417–444s	99–105s	~4.2x
web-unit-tests	117–140s	61–66s	~2.0x
e2e-tests-server-cli	467–476s	290–366s	~1.4–1.6x
docker-build-server (cold)	437s	251s	~1.7x

The standout: ESLint, 5.3x

The “Run linter” step alone took **390s on GitHub Actions vs 73s on Blacksmith** — 5.3x, far beyond the ~2x pattern everywhere else, and it held across all three runs (444/435/417s vs 105/99/101s total job time), so it's not a cache artifact. It makes sense: ESLint's TypeScript-aware rules are largely single-threaded, CPU-bound work — exactly the profile where Blacksmith's bare-metal single-core-performance claim should show its biggest gains, and it did.

Where they didn't differ

Docker layer caching is a wash. Warm rebuilds went 437s→18s on GHA and 251s→17s on Blacksmith — both effectively instant when nothing changed. **Reliability is a genuine tie:** 14 job-runs attempted per platform, one failure each — and it was the *same* failure, my own submodule bug, fixed identically on both sides. No flakiness, no cache corruption, no run that queued and never started. Fair warning: 14 runs says “no flakiness observed,” not “provably equally reliable” — real flake-hunting needs 20–30 runs per cell. And since Blacksmith is a runner substitution, not a different CI product, day-to-day debugging — logs, per-step timing, failure reporting — is literally the same GitHub Actions UI on both (verified with a deliberate broken-assertion push to each branch: identical Vitest output).

The setup tax is real — and one-time

Getting Blacksmith running took real effort that GitHub Actions never asks for: its signup requires the repo to be owned by a GitHub *organization* (I had to transfer both repos out of my personal account mid-experiment, which auto-cancelled two queued runs), there was an overnight wait for Blacksmith's human approval of the App install — the single biggest source of dead time in the whole session — and then a workflow-editing pass to swap runner labels and cache actions. Their docs also never mention the org requirement, and their Migration Wizard defaults to auto-rewriting every workflow in the repo — fine for a real migration, actively wrong for a controlled comparison.

But I've come around on how much this should count against them. Every bit of that friction is **paid once**. For a large engineering org, it's an afternoon plus one approval wait — and then it just runs, with near-zero maintenance, delivering the speedup on every CI run afterward. It's hard to hold a one-time tax against a tool with a permanent dividend. GitHub Actions still wins the "time to first green build" race, clearly. It just doesn't matter much by week two.

My real worry: Blacksmith is invisible to the people it's helping

This is the finding I keep coming back to, and it's not about whether the product works — it does. It's that at no point in this entire pilot did "Blacksmith" surface to a working engineer. Every run, every log, every green check lives in the normal GitHub Actions UI. Nobody sees a Blacksmith logo. Nobody gets told why their build got faster. GitHub is front and center the whole time, quietly taking credit for performance it isn't delivering.

That's a real exposure, and here's the concrete failure mode: eighteen months from now, someone in finance — or a new eng lead running a "what tools do we actually still need" audit — finds a Blacksmith line item. They ask around. Nobody on the team can explain what it does; most don't know it's running at all, because it never once introduced itself. So it gets cut, CI quietly gets 2–5x slower, and nobody connects the two for a quarter. A product whose entire value is invisible-by-design needs to work much harder to stay legible to the org paying for it, and right now Blacksmith doesn't.

Scorecard

Dimension	Read	Call
Performance	1.4–5.3x faster on all six jobs; biggest edge on CPU-bound lint	Blacksmith
Reliability	0 platform failures in 14 job-runs per side; no flakiness observed	Tie
Setup & onboarding	GHA is zero-setup; Blacksmith's org + approval + edits is a real but one-time tax	GHA, narrowly
Observability	Identical logs/UI (same GHA under the hood); Blacksmith advertises SSH debug, untested	Tie
Legibility to the org	Blacksmith never surfaces to the engineers it helps — budget-cut exposure	My open worry

Bottom line

The speed is real, consistent, and biggest exactly where the physics say it should be. The reliability gap is zero at this sample size. The setup tax is real but amortizes to nothing. If I were running a big CI bill, I'd take the trade — and then I'd make sure my team actually knew Blacksmith existed, because the product won't tell them.